Parallel Word Count – Phase 1 Report

1. Problem Statement The goal of this project is to design and implement a parallel word count program that counts the frequency of words in a text dataset. This problem belongs to the Data Analytics / Stream Processing domain, where large volumes of text data are processed to extract useful statistics.

In Phase 1, the focus is on moving from a working sequential version to a shared-memory parallel version using OpenMP, and evaluating its performance on multiple thread configurations.

2. Baseline Design (Sequential Implementation) The sequential program reads a text file (sample.txt), tokenizes it into words, and counts their occurrences using an unordered_map in C++. Timing was measured using chrono::high_resolution_clock.

Measured Sequential Runtime: Average (T■) = 1.5832×10■■ s

3. Parallelization Approach (OpenMP Implementation) - Thread-level parallelism using #pragma omp parallel. - Each thread maintains a local hash map to avoid data races. - Local maps are merged into a global map using a #pragma omp critical section.

4. Performance Profiling

Threads | Parallel Time (s) | Speedup (S = T■/T■) | Efficiency (E = S/p) -------- | ----------------: | ----------------: | ----------------: 1 | 0.0514395 | 0.0003078 | 0.0003078 2 | 0.0322048 | 0.0004916 | 0.0002458 4 | 0.0135224 | 0.0011707 | 0.0002927 8 | 0.0102167 | 0.0015493 | 0.0001937

5. Memory Hierarchy and Cache Analysis Cache tests (perf stat) showed high cache misses for small workloads. Larger inputs are more cache-efficient and reduce synchronization cost.

6. Amdahl’s and Gustafson’s Analysis Amdahl’s Law: S(p) = 1 / ((1-f) + f/p) Gustafson’s Law: S(p) = p - (1-f)(p-1) Scalability improves for larger datasets as synchronization overhead becomes negligible.

7. Reflection and Bottlenecks - Measurement noise for small workloads. - Synchronization overhead during merging. - Sequential file I/O. Improvements: Larger datasets, OpenMP reductions, MPI for distributed scaling.

8. Conclusion Both sequential and OpenMP parallel versions were implemented successfully. The speedup was limited due to small input sizes, but parallelization principles and performance profiling were demonstrated.

9. Deliverables - Source Code (C++) - Report (this document) - Plots: speedup_plot.png, efficiency_plot.png - Demo Video (to be recorded)